

Data Flow Guide

How the dashboard loads, which API calls it makes, where data is kept in the browser, and how a service response becomes grids, charts and KPIs — in plain language.

1

call for the
dashboard definition

6

dataset calls in
parallel, shared by
~9 widgets

0

calls for search,
sort, columns,
30/60/90

1. Loading the dashboard — single or multiple APIs?

Multiple calls, in two steps. Widgets never call APIs themselves — the page does it for them.

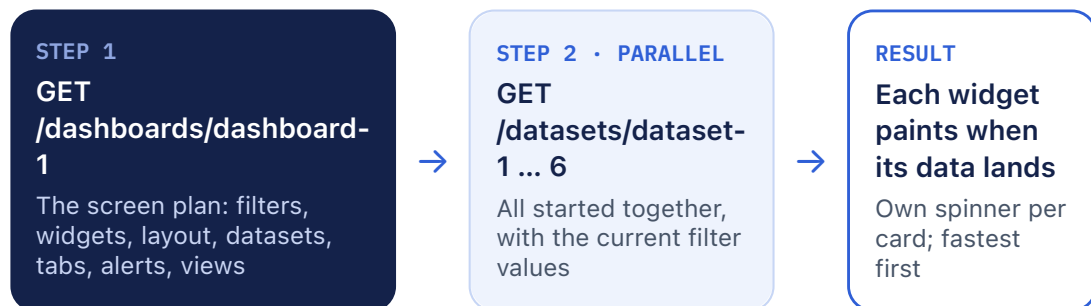


Figure 1. Load sequence. Also: GET /me/navigation once (sidebar) and GET /batches/latest every 15 s (new-data check).

Charts have no separate APIs. They reuse a dataset that is already loaded:

DATASET	KIND	USED BY
dataset-1	grid	Primary grid · KPI cards · "What moved" chart · Daily brief
dataset-2	series	Trend chart · KPI sparklines
dataset-3	grid	FS type grid
dataset-4	series	FS type over time (stacked chart)
dataset-5	grid	Limits grid · "Limits need attention" strip
dataset-6	series	Exposure vs limit chart (loads for the selected limit)

Table 1. One dataset can feed several widgets — one call, one copy in memory.

Why multiple calls instead of one big API

Faster first view

Small calls run in parallel; nothing waits for the slowest query.

Less reloading

A filter or click reloads only the datasets that use it.

Failures stay small

One failed call shows Retry on one card; the rest keep working.

Reusable

A new dashboard is a new definition reusing the same dataset endpoints.

2. What happens on each user action

USER DOES	API CALL?	WHAT HAPPENS
Changes a top filter (Compare, Model, Measure...)	YES · some	Only datasets that use that filter reload; others stay.
Clicks a grid row (drilldown)	YES · linked	The row's rowId is added to the linked datasets (e.g. FS type, trend); only those reload. The clicked grid does not.
Search, sort, show/hide columns	NO	AG Grid works on the rows already in memory (section 3).
30D / 60D / 90D on a chart	NO	The series already holds 90 days; 30D shows the last 30.
Returns to a filter used before	NO	Served instantly from the cache.
New risk batch published	YES · on apply	Banner asks first (or auto-applies, per setting); changed cells flash.

Table 2. Clicks and their cost.

3. How search, sort and columns work without an API

After a dataset call returns, **all rows and all columns are already in the browser**. AG Grid uses its **client-side row model** (our Phase 1 choice): it keeps the full row list in memory and only changes what is *shown*.

Search

Quick filter checks each row in memory for the text and hides non-matches. Clear it — rows return instantly.

Sort

Header click reorders the rows already held. The totals row stays pinned.

Columns

Every column and value was sent. Hiding only stops drawing it; the choice is saved in the browser.

Why it is fast: datasets are small (e.g. 16 rows, 90 days) — a few KB. Sorting a few thousand rows in memory takes milliseconds, with no network. **Limit:** this suits up to tens of thousands of rows; much larger grids move to AG Grid's server-side row model in Phase 2, where search, sort and paging become API calls.

4. Where the data is stored in the browser

In the tab's RAM (JavaScript memory). TanStack Query does not write risk data to disk — no localStorage, IndexedDB or cookies.

- The app creates one `QueryClient` ; inside it a **QueryCache** — essentially an in-memory map.
- Each response is stored under a **query key** = dataset + filter values, e.g. `['dataset', 'dataset-1', 'asOf=2026-09-11&compare=DoD...']` . Another filter combination is a separate entry.

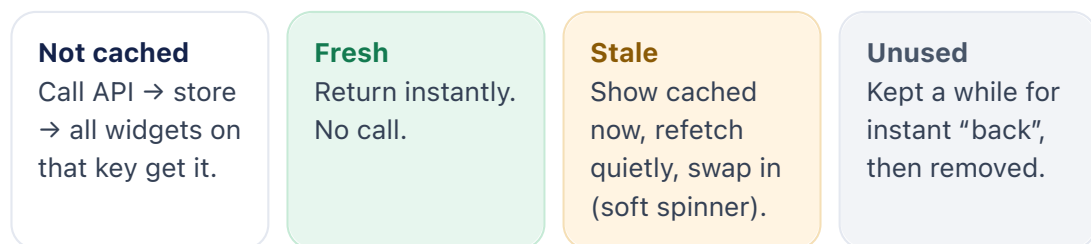


Figure 2. Cache lifecycle. $\text{staleTime} \approx 60 \text{ s}$ (fresh window) · $\text{gcTime} \approx 10 \text{ min}$ (keep unused). Both can be set per dataset.

- **Reload or close the tab** → memory is cleared; next load calls the APIs again. Each tab has its own cache.
- **Saved to disk:** only small preferences via redux-persist (localStorage) — density, sidebar, column choices, saved views. **No risk numbers.**
- **AG Grid and Highcharts** point to the same cached rows; they add only display state. Total memory: a few hundred KB.
- **Optional later:** a persister can keep the cache in IndexedDB for instant reloads. For risk data we recommend memory-only — nothing sensitive left on disk, numbers always current.

5. Is this TanStack Query, or any React app?

Any React app keeps fetched data in browser memory (useState, Redux, variables). TanStack Query turns that memory into a **managed cache**.

CONCERN	PLAIN USESTATE + FETCH	WITH TANSTACK QUERY
Component goes away	Data lost; refetch on return	Kept; instant on return
Three widgets, same data	Up to three identical calls	One call, shared
Freshness / background refresh	Hand-written	staleTime rules built in
Loading · error · retry	Hand-written per component	Ready states → per-card spinner and Retry
Memory clean-up · polling	Hand-written	gcTime · refetchInterval

Table 3. Separate from both: the browser's own HTTP cache, controlled by server headers (Cache-Control). For risk data services should send no-store and let TanStack Query cache in memory.

6. Why the mocks are code, not JSON files

The real API returns different numbers for every filter combination. Compare × Model × Measure × Horizon × Attribute × Region × drilldown = **hundreds of responses**. One function per dataset builds the right one on request.

FILE (SRC/MOCKS)	ROLE
dashboard.ts	Dashboard definition + sidebar navigation
datasets.ts	dataset1(params) ... dataset6(params) — grid and series responses
seed.ts	Seeded numbers — same every run; totals match rows, charts match grids
mockApi.ts	Fake server: routes URLs, adds latency, errors, new batches
servicesPack.ts	Exports every call as JSON for the services team

The output is plain JSON — the same shape services will send. To get files: dev console **Ctrl + Shift + D → Simulate → Download services pack** (one file per call, with drilldown and error examples).

7. Mock or real — how the response arrives

MOCK • `VITE_USE MOCKS ≠ false`

api/client.ts →
mocks/mockApi.ts runs the
matching builder. No network.

REAL • `VITE_USE MOCKS=false`

api/client.ts →
fetch(VITE_API_BASE +
' /datasets/...') on the services.

The rest of the app cannot tell the difference. Switching to real services is a **.env setting — no code change**.

8. How a response becomes the UI

1

Check — validated against the contract (`src/contracts`, `zod`). Wrong shape → the card shows its error state; the console logs exactly what is wrong.

2

Cache — TanStack Query stores it in memory under its dataset key.

3

Deliver — `WidgetHost` hands each widget its dataset, loading status and selection. Widgets never fetch.

4

Draw — each widget type renders the same kind of data its own way (table below).

5

Interact — search, sort and cross-filters run in memory; drilldowns reload only linked datasets.

WIDGET	SERVICE SENDS	UI TURNS IT INTO
Grid	<code>rowData</code> · <code>pinnedBottomRowData</code>	Passed straight to AG Grid (rows + totals row)
	<code>type: ['money']</code>	\$MM format + exact-amount value tooltip
	<code>styleRules</code> · <code>_meta tones</code>	Green / amber / red cell classes, row and cell highlights
	<code>cellRenderer: 'labelCell'</code>	Our React cell component (bars, badges, totals)
	<code>gridOptions</code>	Merged: library defaults → service → consumer → user prefs
Chart	<code>rowData (by date)</code> · <code>seriesDefs</code>	Highcharts series — name, colour index, dash, stack; same rows can feed several charts
KPI	<code>grid totals row</code>	Value + change vs previous + sparkline from a series dataset

Table 4. Services send tones and renderer names — never colours, CSS or code.

IN ONE LINE

One call learns the screen, a few parallel dataset calls feed all widgets, data is kept in memory by TanStack Query, and after that only the affected datasets reload — everything else happens instantly in the browser.